

CS & IT ENGINEERING



'C' Programming

Functions

Lecture No.- 02



By- Satya sir

Recap of Previous Lecture



Functions



- Function Definition
- Declaration vs Definition vs Calling
- Caller, callee
- Function Prototypes

<u>Arg</u>	<u>return values</u>
X	X
X	✓
✓	X
✓	✓

Topics to be Covered



- Scope
 - Static Scoping
 - Dynamic Scoping
- Recursion
 - Types of Recursion



Scope of Variable :

- The extent (or) limit, upto which a variable is visible/accessible.

- 2 types of scope

1) Block scope (or) Local scope

2) Global scope (or) File scope

Block scope : A variable is accessible only within the block, in which it is declared.

Global scope : A variable is accessible throughout Program.

NOTE : Local variable always has priority over Global variable.



Topic : Scope and Lifetime



Ex:

```
void fun(void);  
void main( )
```

```
{  
  int x=5; // local variable to  
            main function
```

```
  x = x+1; ✓
```

```
  x = x*2; ✓
```

```
  printf("%d", x); ✓
```

```
  fun( );
```

```
}
```

```
void fun( )
```

```
{  
  int y; // local variable to fun( )
```

```
  y = x+3; → // Error, x is Not declared
```

```
  printf("%d", y);
```

```
}
```

```
void fun(void);
```

```
int x=10; // Global variable
```

```
void main( )
```

```
{ int x=5; // Local variable to main( )
```

```
  x = x+1;
```

```
  x = x*2;
```

```
  printf("%d", x);
```

```
  fun( );
```

```
}
```

```
void fun( )
```

```
{ int y;
```

```
  y = x+3; // Global 'x' is accessed, as there is no  
            local x to fun( ).
```

```
  printf("%d", y);
```

```
}
```




Topic : Scope and Lifetime



Static Scoping

- Local Variable > Global Variable
- Static Scoping rule is followed in Languages like C, C++, Java, Python, Javascript - - -

Ex:

```
void g(void);  
void h(void);
```

```
int x=10, y=20;
```

```
void f()  
{
```

```
  int x=5, y;
```

```
  y = x * 2; // y=10
```

```
  g();
```

```
  printf("%d", x); // 5
```

```
  h();
```

```
  printf("%d", y); // 10  
}
```

```
void g()  
{
```

```
  int y;
```

```
  y = x * 2; // y=20
```

```
  printf("%d", y); // 20  
}
```

```
void h()  
{
```

```
  int x = 1035;
```

```
  x = x + y; // x=35
```

```
  printf("%d", x); // 35  
}
```

```
void main()  
{
```

```
  int x=2, y=3;
```

```
  f();  
}
```

o/p: 20 5 35 10

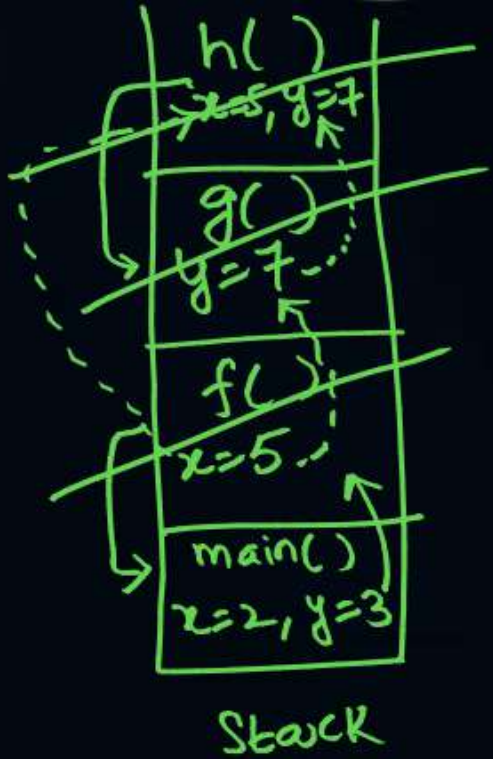


Topic : Recursion



Dynamic Scoping :

- The value of a variable is decided at runtime.
- Dynamic scoping rule is followed by Languages like Lisp, Bash, PERL*.
- Local Variable > Recent invoked/accessed function variable > Global Variable.



Ex:

```
int x=10, y=20; ✓
void f(void);
void g(void);
void h(void);
void f()
{
    int x=5;
    printf("%d %d", x, y); // 5 3
    g();
}
```

```
void g()
{
    int y=7;
    h();
    printf("%d %d", x, y); // 5 7 ✓
}
void h()
{
    printf("%d %d", x, y); // 5 7
}
```

```
→ void main()
{
    int x=2, y=3;
    f();
    printf("%d %d", x, y); // 2 3
}
```




Topic : Recursion



Recursion

- The Process of calling a function by itself is known as Recursion.
- Such function is said to be Recursive function. [caller function == callee function]
- While Implementing Recursion, one need to be careful about Infinite Recursion. Otherwise, Memory overflow / Stack overflow / Segmentation fault occurs.

Ex:

int ~~i~~^{x-1} = 5;

```
Void main()  
{  
    printf("%d", i);  
    i = i - 2;  
    if (i >= 1)  
        main();  
}
```

O/p: 5 3 1

- Recursive function's code can be grouped into 2 sections:

- ① Base Case Section : Indicates the stmt where Recursion terminates.
- ② Recursive Case Section : where Recursive calling stmt(s) exist.
- ③ Remainder Section : other statements



Topic : Recursion

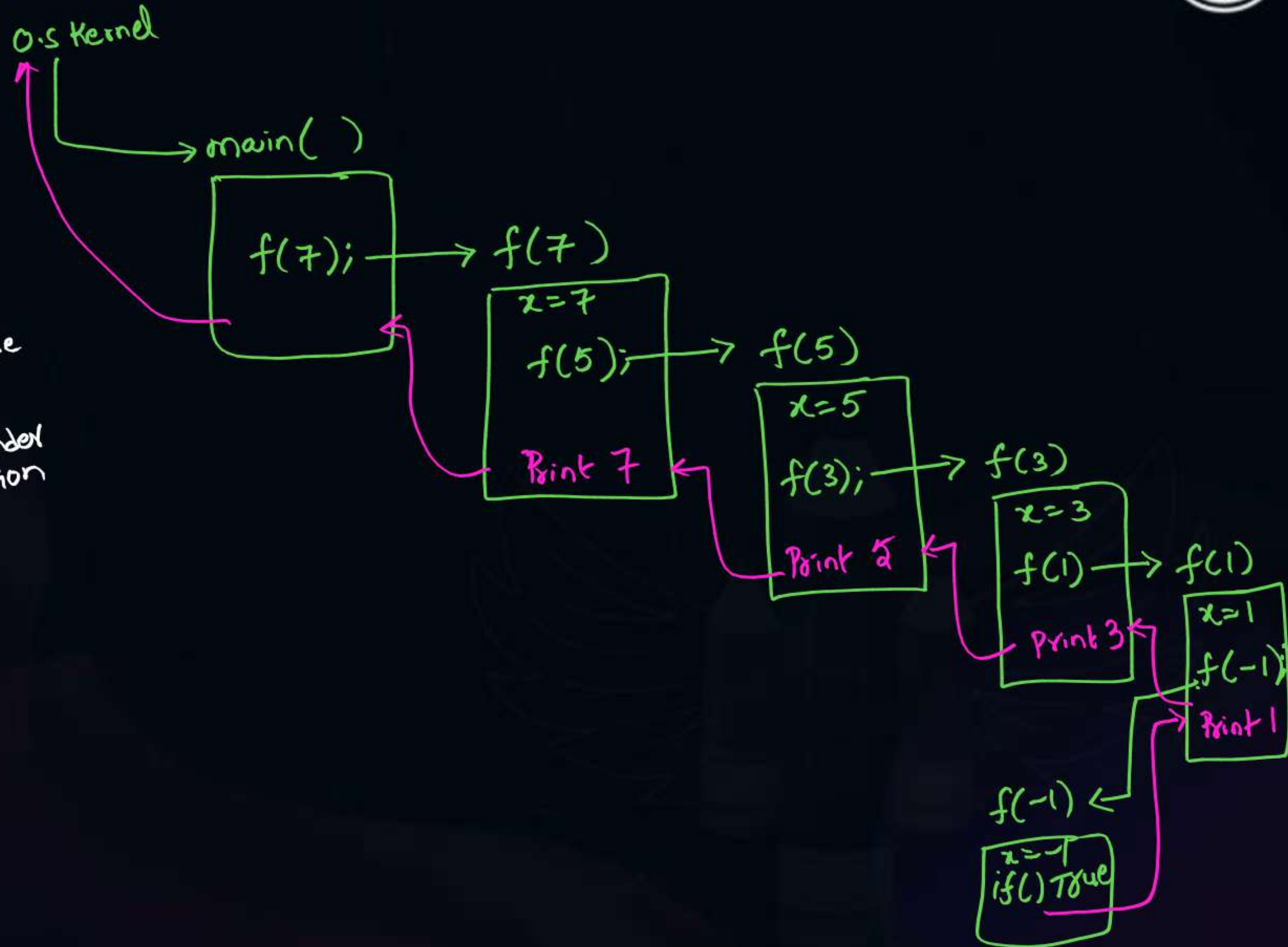


Ex:

```
void f(int x)
{
    if(x < 1)           Base case
        return;
    f(x-2);             Recursive case
    printf("%d", x);     Remainder section
}
```

```
void main()
{
    f(7);
}
```

o/p: 1 3 5 7





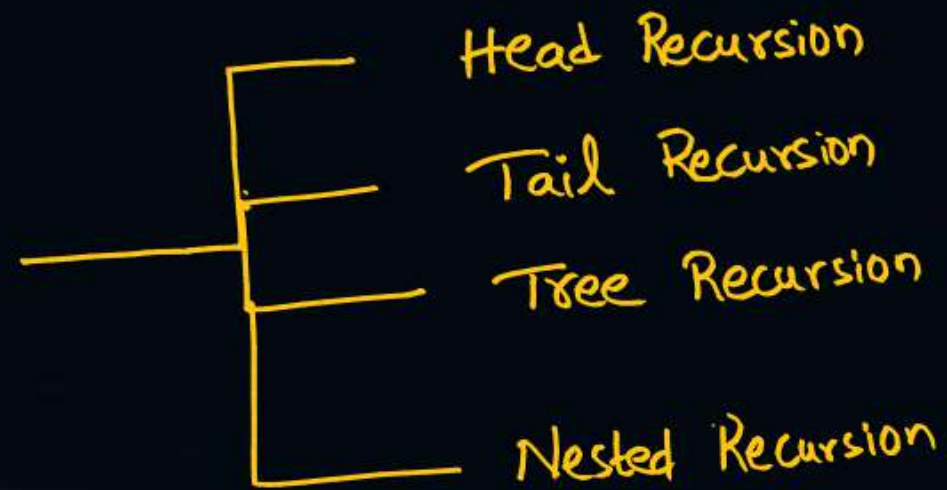
Topic : Recursion



Types of Recursion

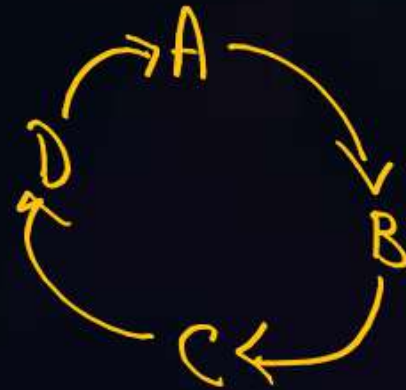
① Direct Recursion

② Indirect Recursion



Direct Recursion: function calls itself directly. $\rightarrow f()$

Indirect Recursion: function calls itself through some other function(s).





Topic : Recursion



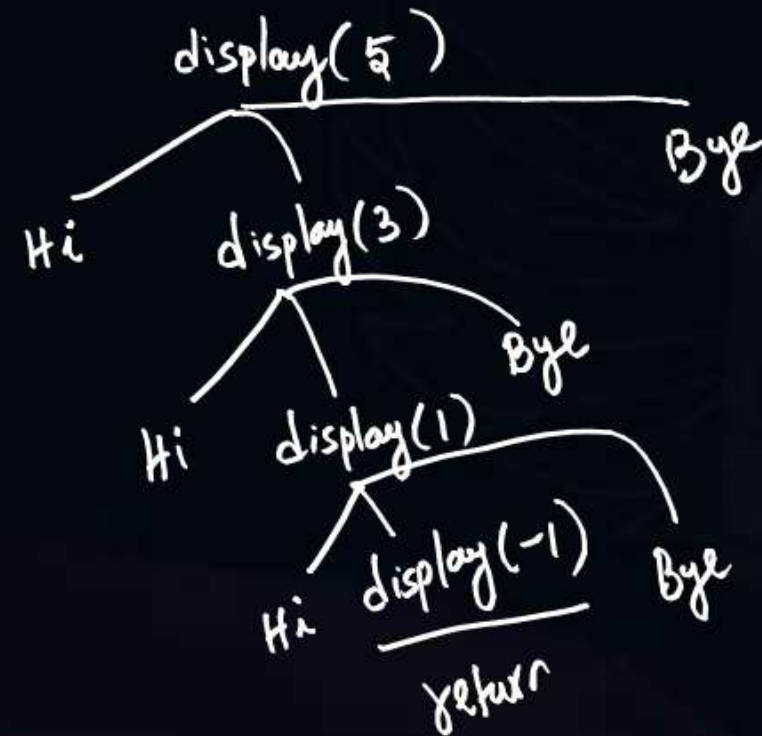
Head Recursion :

- If, after Recursive calling statement, there exists some more statement(s) in the body of function, Then, it is said to be Head Recursion.

- Ex:

```
void display(int i)
{
    if(i <= 0)
        return
    printf("Hi");
    display(i-2);
    printf("Bye");
}
```

o/p if called $\text{display}(5) = \underline{\text{Hi Hi Hi Bye Bye Bye}}$





Topic : Recursion



Tail Recursion

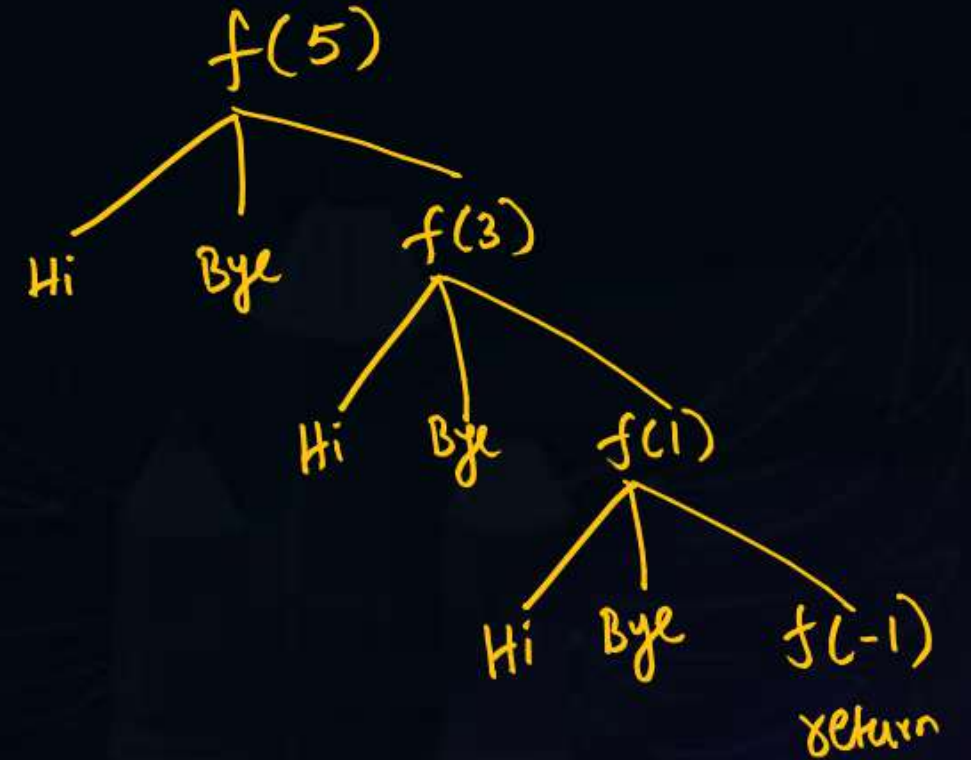
- when Recursive calling Statement/action is the Last One to Execute/Perform, Then, it is Said to be Tail Recursion.

Ex:

```
void f(int x)
{
    if(x <= 0)
        return;
    printf("Hi");
    printf(" Bye");
    f(x-2);
}
```

$f(5)$ o/p = _____

// Hi Bye Hi Bye Hi Bye





2 mins Summary



- Scope
- Static scoping, Dynamic Scoping
- Recursion, Types
 - Head Recursion
 - Tail Recursion

To be Contd ...





THANK - YOU